

MODULE 04 · DOMAIN 1 - FOUNDATION MODEL INTEGRATION, DATA MANAGEMENT, AND COMPLIANCE

Where the Vectors Live

Task 1.4. Choosing a vector store, designing metadata, scaling the index, and keeping it current.

DOMAIN 1 of 5 EXAM WEIGHT 31% MODULE 4 of 22

A vector store does not hold your documents. It holds **chunks**, and what it returns is the entire universe the model gets to reason about.

One retrieval, four steps

01

Embed

The query becomes a vector using the same model as ingestion.

02

Filter

Metadata narrows the candidate set before or during search.

03

Search

Approximate nearest neighbour finds the closest vectors.

04

Return

Top-k chunks with their text, scores and metadata.

ON THE EXAM

The query must be embedded with the same model used at ingestion. Change the embedding model and you must re-embed the entire corpus. There is no partial migration.

What vector search is good and bad at

- ◆ **Good:** paraphrase and concept matching. "Time off" finds "Annual leave".
- ◆ **Bad:** exact identifiers. Part numbers and error codes are what embeddings blur.
- ◆ **Bad:** negation and comparison. "Not signed in 2024" is not a distance.
- ◆ **Bad:** aggregation. "How many claims exceeded £10,000" is a SQL question.
- ◆ Scores are **relative, not calibrated**. A top result exists even when nothing fits.

ON THE EXAM

A vector store always returns its k nearest chunks, so an empty corpus and a perfect match look identical to the calling code. This is why hybrid search exists.

Bedrock Knowledge Bases now come in two types

MANAGED KNOWLEDGE BASE

AWS owns the vector store

- GA 17 June 2026, type MANAGED
- No vector database to provision at all
- Six native connectors, automatic sync
- Document-level access control built in

CUSTOM KNOWLEDGE BASE

You own the vector store

- The original model, still fully supported
- You choose and operate the store
- Full control of index and ANN parameters
- Portable, tunable, and yours to run

THE TIEBREAKER

The exam guide predates the managed type and describes the custom one throughout. Learn both, and answer within whatever options a question puts on the page.

What a Knowledge Base does for you

- ◆ **Parsing.** Documents to text, with Bedrock Data Automation available for layout-heavy PDFs.
- ◆ **Chunking.** Fixed-size, semantic, hierarchical, or none if you chunk yourself.
- ◆ **Embedding.** Titan Text Embeddings V2, Cohere Embed v3 and v4, Nova Multimodal Embeddings.
- ◆ **Retrieval APIs.** *Retrieve* returns chunks. *RetrieveAndGenerate* returns a grounded answer with citations.
- ◆ **Sync.** Ingestion jobs that detect what changed rather than reprocessing everything.

ON THE EXAM

RetrieveAndGenerate is the one-call path and returns citations automatically. *Retrieve* gives you the chunks so you can build the prompt yourself. Questions distinguish them.

Where you can put the vectors

STORE	HERITAGE	CHOOSE IT WHEN
OpenSearch Serverless	Search cluster	The default. Hybrid search, low latency, tunable
OpenSearch Managed Cluster	Search cluster	You already run OpenSearch and want control
S3 Vectors	Object storage	Huge corpora, infrequent queries, cost dominates
Aurora PostgreSQL, pgvector	Relational	Vectors belong beside relational data you already query
Neptune Analytics	Graph	Relationships between entities matter, not just similarity
Pinecone, MongoDB Atlas, Redis	Third party	You already run them

ON THE EXAM

None of these was born a vector database. Each bolted vector search onto an existing engine, and that heritage is what decides which one fits.

Vectors here, documents there, metadata elsewhere

- ◆ The exam guide names **RDS with S3 document repositories** and **DynamoDB with vector databases for metadata and embeddings**.
- ◆ Both describe the same split: the **vector index holds chunks**, and the full document lives somewhere cheaper.
- ◆ Retrieval returns a chunk plus a pointer. The application fetches the whole document only when it needs it.
- ◆ **DynamoDB** suits per-chunk metadata needing single-digit-millisecond lookup by key.
- ◆ Storing whole documents inside the vector index inflates it and slows every search.

ON THE EXAM

You are not obliged to keep everything in one store. Chunks and embeddings in the vector index, documents in S3, hot metadata in DynamoDB is a normal production shape.

OpenSearch Serverless or S3 Vectors?

OPENSEARCH SERVERLESS

Fast, and always running

- Low, consistent query latency
- Hybrid search and rich k-NN tuning
- Billed by OCU with a minimum floor
- You pay for idle capacity

S3 VECTORS

Cheap, and elastic to zero

- Up to 2 billion vectors per index
- Sub-second, around 100 ms when queried often
- Pay for storage and requests only
- Up to 90% lower total cost at scale

THE TIEBREAKER

Query rate decides it. Constant interactive traffic pays for OpenSearch happily. A large archive queried a few hundred times a day should not be renting a cluster.

A law firm is building semantic search over 40 years of case files, around 900 million chunks once embedded. Searches are run by 60 solicitors and average roughly 400 queries a day, clustered around mornings.

A three-second response is acceptable. The firm has costed a provisioned OpenSearch Serverless collection and found the monthly figure difficult to justify for the query volume. The corpus grows slowly and is queried far more rarely than it is stored.

Which vector store should the firm choose?

- ☐ A Amazon OpenSearch Serverless, accepting the OCU floor as the cost of low latency
- ☐ B Amazon S3 Vectors as the vector store for the knowledge base
- ☐ C Aurora PostgreSQL with pgvector, scaled to a low minimum ACU
- ☐ D Amazon Neptune Analytics, since case files reference one another heavily

◆ PAUSE AND DECIDE

Which vector store should the firm choose?

A OpenSearch Serverless

Buys low latency the firm has said it does not need, and pays a per-hour floor for 400 queries. The cost objection is stated in the scenario.

C Aurora pgvector at low ACU

Cheaper than OpenSearch, and 900 million vectors is a demanding load for pgvector. You would also own index maintenance and ANN parameter tuning.

B S3 Vectors

Built for exactly this shape: enormous corpus, sparse queries, latency tolerance. Storage-and-request billing matches a workload that is stored far more than queried.

D Neptune Analytics

Graph storage answers relationship questions. The requirement here is semantic search, and cross-referencing is scenery rather than a stated requirement.

THE SIGNAL

Three numbers decide it: 900 million chunks, 400 queries a day, three seconds acceptable. Huge, sparse and latency-tolerant is the exact profile S3 Vectors was built for.

When the answer is not OpenSearch

- ◆ **Aurora pgvector** when the filter is relational. Joining vectors to customers, orders or entitlements in one query.
- ◆ It also wins when the team already runs Postgres. You own *ANALYZE*, vacuum and the HNSW or IVFFlat parameters.
- ◆ **Neptune Analytics** when relationships carry meaning. This is GraphRAG.
- ◆ Graph retrieval answers multi-hop questions: which suppliers connect to a sanctioned entity through two intermediaries.
- ◆ Pure similarity search cannot traverse. It finds documents that *sound* alike, never entities that are *linked*.

ON THE EXAM

The GraphRAG signal is a question no single chunk can answer, because the answer lives in the connections between chunks rather than inside any one of them.

A pharmaceutical company has a knowledge base over research papers, clinical trial records and researcher profiles. Semantic search works well for questions such as "what does the literature say about compound X".

Researchers now want to ask questions such as "which trials were led by investigators who previously worked on compounds in the same therapeutic class as X". No single document contains that answer; it must be assembled by following links between papers, trials and people. Existing semantic search returns plausible but incomplete results for these questions.

What should the company change?

- ☐ A Increase the number of chunks returned so more documents reach the model
- ☐ B Move the knowledge base to Amazon Neptune Analytics to combine graph traversal with vector search
- ☐ C Fine-tune the foundation model on the research corpus so it learns the relationships
- ☐ D Add a reranker to improve the ordering of retrieved chunks

◆ PAUSE AND DECIDE

What should the company change?

A Return more chunks

More of the wrong kind of result. The answer is not sitting in an unretrieved chunk; it does not exist in any single chunk.

C Fine-tune on the corpus

Module 2's rule holds. This is a knowledge and structure problem, and training does not reliably encode facts or relationships.

B Neptune Analytics for graph plus vector

Multi-hop questions need traversal. Graph retrieval follows paper to investigator to earlier compound, which similarity search has no mechanism to do.

D Add a reranker

Reordering improves which of the retrieved chunks appears first. If none of them contains the answer, better ordering changes nothing.

THE SIGNAL

"No single document contains that answer" is the GraphRAG signal stated almost verbatim. When the answer lives between documents, no amount of top-k tuning will find it.

Metadata is what turns a vector store into a system you can secure, filter and keep current. Designing it badly is **expensive to undo**, because fixing it means re-ingesting everything.

What to attach to every chunk

- ◆ **Tenant or owner.** The identifier that decides who may see this chunk. Non-negotiable in multi-tenant systems.
- ◆ **Source and version.** Document ID, URI and revision, so answers can cite and so stale chunks can be replaced.
- ◆ **Timestamps.** Created and modified, so recency can be a filter rather than a hope.
- ◆ **Domain classification.** Department, product line, jurisdiction. This is what narrows the search space.
- ◆ **Authorship and status.** Who wrote it, and whether it is draft, approved or superseded.

ON THE EXAM

The exam guide names S3 object metadata for timestamps, custom attributes for authorship and tagging systems for domain classification. Those three are its examples of this list.

Four things metadata filtering buys

- ◆ **Precision.** Filtering to one department before searching beats searching everything and hoping.
- ◆ **Access control.** A filter on tenant is how one index safely serves many customers.
- ◆ **Recency.** Excluding superseded versions stops the model quoting a policy that was replaced.
- ◆ **Cost.** A narrower candidate set means fewer chunks retrieved and fewer tokens sent.
- ◆ Over-filtering returns nothing, and a vector store returning nothing looks the same as a corpus with no answer.

ON THE EXAM

Access control by metadata filter is the standard multi-tenant answer, and it only works if the filter is applied server-side by trusted code, never chosen by the caller.

A SaaS company serves 800 corporate customers from one Bedrock knowledge base. Every chunk carries a `tenantId` attribute. The web application passes a metadata filter on `tenantId` with each Retrieve call, and the filter value is taken from a parameter in the client request.

A penetration test found that altering that parameter in the browser returned another customer's documents. The company needs isolation it can demonstrate to auditors, and wants the smallest change that removes the vulnerability.

What should the company do?

- A Create a separate knowledge base and vector index per customer, 800 in total
- B Derive `tenantId` server-side from the authenticated identity and apply the filter in trusted backend code, never from client input
- C Validate the `tenantId` parameter against a list of known tenant identifiers before passing it to Retrieve
- D Apply a Bedrock guardrail to block responses containing other customers' data

◆ PAUSE AND DECIDE

What should the company do?

- A** 800 separate knowledge bases
Genuinely isolates, and is a large architectural change with 800 indexes and 800 sync pipelines to operate. Far more than the smallest fix.
- C** Validate the parameter against known tenants
The attacker's substituted value is a valid tenant ID. Validation confirms the value is well-formed, not that it belongs to this caller.

- B** Derive tenantId from the authenticated identity server-side
The vulnerability is that the caller chooses their own filter. Deriving it from the verified identity removes the choice entirely.
- D** Guardrail on responses
Guardrails filter content categories, not ownership. They have no way to know which customer a given document belongs to.

THE SIGNAL

Metadata filtering is only an access control if the filter is not attacker-controlled. C is the sharpest distractor: it validates the format of the value while leaving the caller free to choose it.

Making the index fast at scale

- ◆ **HNSW** is the usual graph index. Fast queries, high memory, slower builds.
- ◆ **IVF** clusters vectors. Less memory, recall depends on clusters probed.
- ◆ **Dimensions cost memory linearly**. Titan V2 supports reduced dimensions.
- ◆ **Shards** add parallelism. Too many adds coordination overhead.
- ◆ All approximate search trades **recall for speed**. Tuning moves you along that curve.

ON THE EXAM

'Approximate' in approximate nearest neighbour is a promise, not a caveat. Exact search over a billion vectors is not realistic, so every design accepts missed results.

One index with filters, or many indexes?

ONE INDEX, FILTERED

Simple to operate

- One ingestion pipeline and one schema
- Cross-tenant analytics stay possible
- Filter correctness becomes security-critical
- A noisy tenant affects everyone's latency

INDEX PER DOMAIN OR TENANT

Isolated, and more to run

- Hard isolation with no filter to get wrong
- Smaller indexes search faster
- Per-tenant tuning and lifecycle
- N pipelines, N schemas, N things to break

THE TIEBREAKER

Split when isolation is a legal or contractual requirement, or when domains have genuinely different retrieval behaviour. Do not split merely because you have many tenants.

An engineering firm runs a single OpenSearch Serverless index holding 200 million chunks spanning twelve unrelated product lines. Retrieval latency at the 95th percentile has risen to 1.8 seconds, and users report that answers about one product sometimes cite documents from a completely different one.

Each query is known to concern exactly one product line, and that product line is already recorded as a metadata attribute on every chunk. The application currently sends no filter with its retrieval calls.

What is the most effective change?

- | | |
|--|---|
| ☐ A Apply a metadata filter on product line to every retrieval call | ☐ B Split the index into twelve separate indexes, one per product line |
| ☐ C Increase the OpenSearch Serverless OCU allocation to reduce latency | ☐ D Reduce the embedding dimension count to shrink the index in memory |

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Filter on product line at query time
Fixes both symptoms at once and costs one line of code. The attribute already exists, so no re-ingestion is required.
- B

 Twelve separate indexes
Would also fix both, by re-ingesting 200 million chunks and building twelve pipelines. Correct outcome, enormously more work.
- C

 More OCUs
Buys latency and does nothing about cross-product citations. Paying more to search irrelevant data faster.
- D

 Fewer embedding dimensions
Shrinks memory and requires re-embedding everything, while reducing retrieval quality. It treats a filtering problem as a capacity problem.

THE SIGNAL

Two symptoms, one cause: the search space is twelve times larger than it needs to be. "Already recorded as a metadata attribute" is the scenario telling you the fix costs nothing.

Connecting to where the content actually lives

- ◆ **Managed KB connectors:** S3, SharePoint, Confluence, Google Drive, OneDrive and Web Crawler.
- ◆ **Custom KB connectors:** S3, SharePoint, Confluence, Salesforce, Web Crawler, and a Custom connector.
- ◆ The **Custom connector** is the escape hatch. You push chunks in directly for anything without a native connector.
- ◆ Managed Knowledge Base offers **document-level access control** for the enterprise connectors.
- ◆ That access control does **not** apply to the Web Crawler, which has no identity to map permissions from.

ON THE EXAM

Kendra shipped over thirty connectors; Knowledge Bases ship a handful. If a scenario names an exotic source system, the answer is the Custom connector plus your own extraction.

Stopping the index going stale

- ◆ **Ingestion jobs sync incrementally.** Changed and new documents only, not the whole corpus.
- ◆ **Deletions must propagate.** A deleted source document whose chunks remain will still be retrieved and cited.
- ◆ **Event-driven:** S3 Event Notifications or EventBridge start a sync the moment content changes.
- ◆ **Scheduled:** a fixed refresh cadence. Simpler, and stale between runs.
- ◆ A knowledge base with no sync strategy at all degrades invisibly. Nothing errors; the answers just quietly become wrong.

ON THE EXAM

The deletion case is the one people miss. Adding and updating are obvious; removing chunks for a retracted document is what stops a system citing a policy that was withdrawn.

Event-driven or scheduled?

EVENT-DRIVEN

Fresh within seconds

- S3 notifications or EventBridge trigger sync
- Needed when staleness has consequences
- More moving parts and more sync jobs
- Bursty updates can trigger sync storms

SCHEDULED

Fresh within hours

- One cadence, one job, easy to reason about
- Fine when content changes slowly
- Predictable ingestion cost
- Guaranteed stale window between runs

THE TIEBREAKER

Ask what happens if an answer is four hours old. For a product catalogue, nothing. For a drug interaction warning or a trading limit, quite a lot.

A retailer's support assistant answers questions from a knowledge base built over product and policy documents in Amazon S3, synchronised nightly. Agents have begun reporting that the assistant quotes a returns policy withdrawn three weeks ago, and occasionally recommends discontinued products.

Investigation shows the nightly ingestion job completes successfully every night. The withdrawn policy document was deleted from S3 at the time of withdrawal, and the discontinued products were removed from the catalogue file.

What is the most likely cause and the correct fix?

- ☐ A The embedding model has drifted; re-embed the corpus with a newer model
- ☐ B Deletions are not propagating to the vector index; ensure the sync removes chunks for deleted source documents and verify with a retrieval test
- ☐ C Nightly sync is too infrequent; move to event-driven synchronisation
- ☐ D Retrieval is returning too many chunks; reduce top-k so older documents rank out

What is the most likely cause and the correct fix?

A Embedding drift; re-embed

Embedding models do not drift. They are fixed functions. Nothing in the symptoms points at embedding quality anyway.

B Deletions not propagating; fix removal and verify

The job succeeds nightly, so additions and updates work. Only removals are failing, which is exactly the observed symptom.

C Move to event-driven sync

Would help if content were newer than the last run. This content is three weeks old, so twenty-one successful syncs have already passed over it.

D Reduce top-k

Retrieves fewer chunks and does not remove the wrong ones. A deleted document that ranks first will still rank first.

THE SIGNAL

"Three weeks ago" plus "the job completes successfully every night" is the whole diagnosis. Twenty-one successful syncs have run, so frequency is not the problem. Removal is.

Where this module is ahead of the exam

- ◆ **Managed Knowledge Base** is from June 2026. Question banks written earlier assume you always bring a vector store.
- ◆ **S3 Vectors** reached GA in December 2025. Older material offers only OpenSearch, Aurora and third parties.
- ◆ If neither appears in the options, **answer within the set on the page**. OpenSearch Serverless is the safe default.
- ◆ **Amazon Kendra** still appears as a correct answer for enterprise search, despite closing to new customers in July 2026.
- ◆ The underlying decisions are stable: query rate, filter shape, relationship depth, and who operates the store.

ON THE EXAM

Both directions of drift now apply to this domain. The exam names services you cannot buy, and this course names services the exam may not know about.

The five things to carry forward

- ◆ **Query rate, not corpus size**, decides OpenSearch versus S3 Vectors.
- ◆ **Aurora pgvector** when the filter is relational. **Neptune Analytics** when the answer lives between documents.
- ◆ **Design metadata before ingestion**. Tenant, source, version, timestamp, domain. Fixing it later means re-ingesting.
- ◆ A metadata filter is only access control if the **caller cannot choose it**. Derive it from the authenticated identity.
- ◆ **Deletions must propagate**. A stale index fails silently, with confident citations of documents that no longer exist.

ON THE EXAM

Module 5 takes the other half of RAG: chunking, embedding models, hybrid search, rerankers and query transformation. That is where retrieval quality is actually won.